Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №7

“Робота з базами даних у Python”

Виконав:
студент 3 курсу
групи ФЕП-33с
Качмар Денис

Викладач:
Проф. Свелеба Сергій Андрійович

Львів - 2026

ХІД РОБОТИ

Тема: Робота з базами даних у Python

Мета: Ознайомитися з основними принципами організації шару зберігання даних (persistence layer) у Python-застосунках, навчитися працювати з реляційними базами даних, мовою SQL, СУБД SQLite, стандартом Python DB-API (PEP 249), параметризованими запитами, CRUD-операціями, транзакціями, принципами ACID, патерном Repository, SQLAlchemy ORM (версії 2.0+) та міграціями Alembic. Навчитися інтегрувати persistence layer у Python-проєкт, створений у попередніх лабораторних роботах, та перевіряти його за допомогою автоматизованих integration tests на окремій тестовій базі даних.

Варіант: 5 (Каталог автомобілів)

Постановка задачі: Продовжити проєкт попередніх лабораторних робіт без зміни предметної області. Для автомобіля забезпечити збереження параметрів: ID, марка (виробник), модель, рік випуску, ціна, пробіг та унікальний міжнародний номер кузова (VIN). Створити пов'язані реляційні таблиці `manufacturers` та `cars` зі зв'язком one-to-many із каскадним видаленням (ON DELETE CASCADE). Реалізувати репозиторії `ManufacturerRepository` та `CarRepository` (Repository Pattern), відокремивши persistence layer від бізнес-логіки. Зберегти функціональність Lab_01: пошук за маркою, фільтрація за роком, найдорожчий автомобіль, середня ціна, мінімальний пробіг; додати повний CRUD, зміну ціни та пробігу. Через Python DB-API реалізувати параметризований пошук автомобілів за маркою/виробником. Реалізувати транзакційну операцію: масове оновлення цін для вибраного виробника з фіксацією (commit) та відкатом (rollback) у разі некоректних даних чи перевищення допустимого ліміту. Налаштувати Alembic та створити міграцію додавання обов'язкового поля `vin` з обмеженням UNIQUE. Створити автоматизовані integration tests на ізольованій тестовій базі SQLite (`:memory:`): CRUD, зв'язок виробник-авто, фільтрація за роком, агрегація даних (COUNT, AVG, MIN, MAX), перевірка UNIQUE VIN та rollback транзакції при збої.

---

1. Опис предметної області та сутностей

Предметна область системи — облік та аналітика каталогу легкових транспортних засобів. Ключовими сутностями предметної області є: - **Виробник (Manufacturer):** автомобільний бренд/концерн, що випускає автотранспортні засоби (наприклад, Toyota, BMW, Porsche). Характеризується унікальною назвою бренду та країною походження. - **Автомобіль (Car):** конкретний екземпляр автотранспорту в каталозі. Характеризується моделлю, роком випуску, ціною в USD, пробігом у км та унікальним міжнародним номером

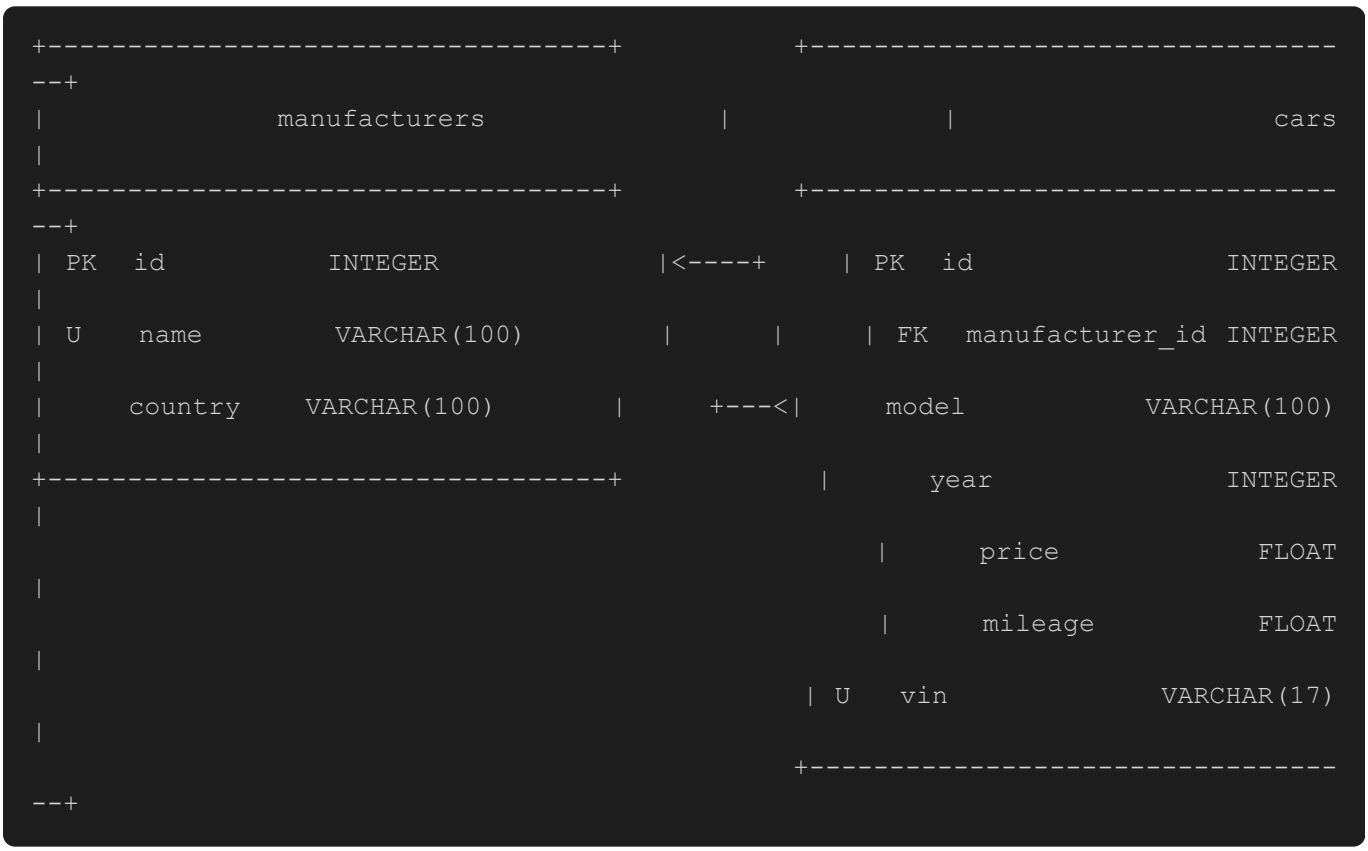
кузова VIN (Vehicle Identification Number). Кожен автомобіль жорстко асоційований із конкретним виробником (`one-to-many`).

Ключові бізнес-правила системи: 1. Назва автовиробника є глобально унікальною в каталозі. 2. При видаленні виробника всі зареєстровані за ним автомобілі підлягають автоматичному каскадному видаленню (`ON DELETE CASCADE`), що унеможливлює появу осиротілих записів (dangling references). 3. Рік випуску автомобіля обмежений історичним діапазоном (від 1886 року — створення першого авто Карлом Бенцом — до поточного року + 1). 4. Ціна та пробіг не можуть бути від'ємними значеннями. 5. VIN-код автомобіля складається з 17 символів та є суворо унікальним у всій базі даних.

---

2. Схема бази даних та зв'язки (ER-діаграма)

Реляційна схема містить відношення «один-до-багатьох» (1:N) між таблицями `manufacturers` та `cars`:



- Зв'язок: `manufacturers.id (1) <----> (N) cars.manufacturer_id`.
- Обмеження посилальної цілісності: `FOREIGN KEY (manufacturer_id) REFERENCES manufacturers(id) ON DELETE CASCADE`.

---

3. Таблиці, опис полів, Primary Keys та Foreign Keys

База даних SQLite містить 3 реляційні таблиці: 1. `manufacturers` — реєстр компаній-автовиробників. 2. `cars` — реєстр автомобілів у каталозі. 3. `alembic_version` — системна таблиця версій міграцій Alembic.

Таблиця `manufacturers`

Поле	Тип SQLite	Тип SQLAlchemy	Ключ / Обмеження	Опис
`id`	INTEGER	`Integer`	PRIMARY KEY, AUTOINCREMENT	Первинний числовий сурогатний ключ
`name`	VARCHAR(100)	`String(100)`	NOT NULL, UNIQUE, INDEX	Офіційна назва бренду виробника
`country`	VARCHAR(100)	`String(100)`	NOT NULL, DEFAULT 'Unknown'	Країна походження виробника

Таблиця `cars`

Поле	Тип SQLite	Тип SQLAlchemy	Ключ / Обмеження	Опис
`id`	INTEGER	`Integer`	PRIMARY KEY, AUTOINCREMENT	Первинний числовий ключ автомобіля
`manufacturer_id`	INTEGER	`Integer`	NOT NULL, FOREIGN KEY	Зовнішній ключ (`manufacturers.id`, CASCADE)
`model`	VARCHAR(100)	`String(100)`	NOT NULL	Конкретна комерційна назва моделі
`year`	INTEGER	`Integer`	NOT NULL, CHECK(year >= 1886)	Рік виготовлення автомобіля
`price`	REAL / FLOAT	`Float`	NOT NULL, CHECK(price >= 0.0)	Ринкова вартість у доларах США
`mileage`	REAL / FLOAT	`Float`	NOT NULL, CHECK(mileage >= 0.0)	Експлуатаційний пробіг у кілометрах
`vin`	VARCHAR(17)	`String(17)`	NOT NULL, UNIQUE, INDEX	Унікальний 17-значний номер кузова

Обґрунтування Primary Keys та Foreign Keys:

- **Primary Keys (`manufacturers.id`, `cars.id`):** Забезпечують безпосередній доступ до рядка за O(1) за рахунок автоматично створеного B-Tree індексу.
- **Foreign Key (`cars.manufacturer_id`):** Гарантує посилальну цілісність. При активації `PRAGMA foreign_keys = ON;`, SQLite блокує вставку авто з неіснуючим брендом, а видалення запису з `manufacturers` каскадно видаляє всі підпорядковані авто.

---

4. SQL-команди створення таблиць (DDL)

```
CREATE TABLE manufacturers (
    id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(100) NOT NULL UNIQUE,
    country VARCHAR(100) NOT NULL DEFAULT 'Unknown'
);

CREATE TABLE cars (
    id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
    manufacturer_id INTEGER NOT NULL,
    model VARCHAR(100) NOT NULL,
    year INTEGER NOT NULL,
    price FLOAT NOT NULL,
    mileage FLOAT NOT NULL,
    vin VARCHAR(17) NOT NULL UNIQUE,
       FOREIGN KEY (manufacturer_id) REFERENCES manufacturers (id) ON DELETE
CASCADE,
    CHECK (year >= 1886),
    CHECK (price >= 0.0),
    CHECK (mileage >= 0.0)
);

CREATE INDEX ix_cars_manufacturer_id ON cars (manufacturer_id);
CREATE INDEX ix_cars_vin ON cars (vin);
```

---

5. Параметризований запит через DB-API (PEP 249)

Стандарт Python DB-API вимагає використання параметризованих запитів (Prepared Statements) для відокремлення коду команди від даних. У модулі `src/car_catalog/repositories.py` реалізовано низькорівневу DB-API функцію:

```
import sqlite3

def find_cars_by_manufacturer_dbapi(
    database_path: str,
    manufacturer_name: str,
) -> list[tuple]:
    with sqlite3.connect(database_path) as connection:
        cursor = connection.cursor()
        cursor.execute(
            """
            SELECT c.id, m.name, c.model, c.year, c.price, c.mileage, c.vin
            FROM cars c
            JOIN manufacturers m ON c.manufacturer_id = m.id
            WHERE m.name = ?
            ORDER BY c.id ASC
            """,
            (manufacturer_name,),
        )
        return cursor.fetchall()
```

Використання плейсхолдера `?` передає значення `(manufacturer_name,)` СУБД окремо від тіла запиту, що повністю запобігає загрозі SQL Injection.

---

6. SQLAlchemy ORM-моделі

Моделі описано в src/car_catalog/db_models.py із застосуванням Declarative Mapping SQLAlchemy 2.0+:

```
from __future__ import annotations
from typing import Any
from sqlalchemy import CheckConstraint, Float, ForeignKey, Integer, String
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship

class Base(DeclarativeBase):
    pass

class Manufacturer(Base):
    __tablename__ = "manufacturers"

             id:   Mapped[int]   =   mapped_column(Integer,   primary_key=True,
autoincrement=True)
     name: Mapped[str] = mapped_column(String(100), unique=True, nullable=False,
index=True)
          country:   Mapped[str]   =   mapped_column(String(100),   nullable=False,
default="Unknown")

    cars: Mapped[list[CarEntity]] = relationship(
        "CarEntity",
        back_populates="manufacturer",
        cascade="all, delete-orphan",
        passive_deletes=True,
    )

class CarEntity(Base):
    __tablename__ = "cars"
    __table_args__ = (
        CheckConstraint("year >= 1886", name="check_min_year"),
        CheckConstraint("price >= 0.0", name="check_positive_price"),
        CheckConstraint("mileage >= 0.0", name="check_positive_mileage"),
    )

             id:   Mapped[int]   =   mapped_column(Integer,   primary_key=True,
autoincrement=True)
    manufacturer_id: Mapped[int] = mapped_column(
        Integer,
        ForeignKey("manufacturers.id", ondelete="CASCADE"),
        nullable=False,
        index=True,
    )
    model: Mapped[str] = mapped_column(String(100), nullable=False)
    year: Mapped[int] = mapped_column(Integer, nullable=False)
    price: Mapped[float] = mapped_column(Float, nullable=False)
    mileage: Mapped[float] = mapped_column(Float, nullable=False)
      vin: Mapped[str] = mapped_column(String(17), unique=True, nullable=False,
index=True)

           manufacturer:   Mapped[Manufacturer]   =   relationship("Manufacturer",
back_populates="cars")
```

7. Модуль керування базою даних (database.py)

Модуль src/car_catalog/database.py інкапсулює життєвий цикл пулу з'єднань, активацію Foreign Keys та контекстний менеджер транзакційних сесій:

```
from collections.abc import Generator
from contextlib import contextmanager
from sqlalchemy import Engine, create_engine, event
from sqlalchemy.orm import Session, sessionmaker
from car_catalog.db_models import Base

DATABASE_URL = "sqlite:///cars.db"

def  configure_sqlite_foreign_keys(dbapi_connection:  object,  connection_record:
object) -> None:
    cursor = getattr(dbapi_connection, "cursor", None)
    if cursor is not None:
        raw_cursor = cursor()
        raw_cursor.execute("PRAGMA foreign_keys = ON;")
        raw_cursor.close()

def create_database_engine(database_url: str = DATABASE_URL, echo: bool = False)
-> Engine:
    engine = create_engine(database_url, echo=echo)
    if database_url.startswith("sqlite"):
        event.listen(engine, "connect", configure_sqlite_foreign_keys)
    return engine

def create_session_factory(engine: Engine) -> sessionmaker[Session]:
    return sessionmaker(bind=engine, autoflush=False, expire_on_commit=False)

@contextmanager
def  session_scope(session_factory:  sessionmaker[Session])  ->  Generator[Session,
None, None]:
    session = session_factory()
    try:
        yield session
        session.commit()
    except Exception:
        session.rollback()
        raise
    finally:
        session.close()

def init_database(engine: Engine) -> None:
    Base.metadata.create_all(engine)

def drop_database(engine: Engine) -> None:
    Base.metadata.drop_all(engine)
```

---

8. Реалізація Repository Pattern (CRUD-операції)

У src/car_catalog/repositories.py реалізовано репозиторії: - `ManufacturerRepository`: методи `add`, `get_by_id`, `get_by_name`, `get_all`, `delete`. - `CarRepository`: методи `add`, `get_by_id`, `get_by_vin`, `get_all`, `find_by_manufacturer_name`, `filter_by_year_range`, `get_most_expensive_car`, `get_lowest_mileage_car`, `get_average_price`, `update_price`, `update_mileage`, `delete`, `get_manufacturer_statistics`.

Приклад виконання CRUD-операцій:

```
# 1. Create (Створення)
manufacturer = manufacturer_repo.add(Manufacturer(name="Toyota",
country="Japan"))
car = car_repo.add(CarEntity(
    manufacturer_id=manufacturer.id,
    model="Camry",
    year=2021,
    price=26000.0,
    mileage=50000.0,
    vin="4T1B11HK5MU123456",
))

# 2. Read (Читання)
loaded_car = car_repo.get_by_id(car.id)
assert loaded_car.model == "Camry"
assert loaded_car.manufacturer.name == "Toyota"

# 3. Update (Оновлення ціни та пробігу)
car_repo.update_price(car.id, 24500.0)
car_repo.update_mileage(car.id, 48000.0)

# 4. Delete (Видалення)
deleted = car_repo.delete(car.id)
assert deleted is True
assert car_repo.get_by_id(car.id) is None
```

---

9. Транзакційні операції та принципи ACID (commit та rollback)

Для демонстрації транзакційної цілісності у `src/car_catalog/repositories.py` реалізовано операцію пакетного перерахунку цін автомобілів:

```
def bulk_update_prices_by_manufacturer(
    session: Session,
    manufacturer_id: int,
    multiplier: float,
    max_price_limit: float = 200_000.0,
) -> None:
    if multiplier <= 0:
        raise ValueError("Multiplier must be positive")
    try:
                statement = select(CarEntity).where(CarEntity.manufacturer_id ==
manufacturer_id)
        cars = list(session.scalars(statement))
        if not cars:
                        raise ValueError(f"No cars found for manufacturer ID
{manufacturer_id}")

        for car in cars:
            new_price = round(car.price * multiplier, 2)
            if new_price > max_price_limit:
                raise ValueError(
                                f"Car ID {car.id} exceeds max allowed price limit
{max_price_limit}"
                )
            car.price = new_price

        session.commit()
    except Exception:
        session.rollback()
        raise
```

Пояснення commit, rollback та властивостей ACID:

- `commit()`: Фіксує всі накопичені в транзакції зміни, записує їх у WAL-файл бази та знімає блокування.
- `rollback()`: Повністю скасовує всі непідтверджені зміни у буфері, відновлюючи базу до стану перед початком транзакції.
- **Atomicity:** Якщо хоча б одне авто перевищує `max_price_limit`, спрацьовує `rollback()`, і жодне авто не отримує змінену ціну.
- **Consistency:** Перевіряються обмеження `CHECK(price >= 0)` та `CHECK(year >= 1886)`.
- **Isolation:** SQLite ізолює транзакцію через блокування та WAL-журнал.
- **Durability:** Підтверджені дані зберігаються на фізичному диску.

---

10. Версійні міграції схеми бази даних (Alembic)

Налаштовано конфігурацію alembic.ini та середовище migrations/env.py. Створено дві ревізії: 1. `0001_initial_tables.py`: створення базових таблиць `manufacturers` та `cars` (без поля VIN). 2. `0002_add_vin_to_cars.py`: додавання поля `vin` з `UNIQUE constraint`.

Код міграції додавання поля VIN (migrations/versions/0002_add_vin_to_cars.py):

```
def upgrade() -> None:
    with op.batch_alter_table("cars", schema=None) as batch_op:
                    batch_op.add_column(sa.Column("vin",   sa.String(length=17),
nullable=False))
        batch_op.create_unique_constraint("uq_cars_vin", ["vin"])
        batch_op.create_index("ix_cars_vin", ["vin"])

def downgrade() -> None:
    with op.batch_alter_table("cars", schema=None) as batch_op:
        batch_op.drop_index("ix_cars_vin")
        batch_op.drop_constraint("uq_cars_vin", type_="unique")
        batch_op.drop_column("vin")
```

---

11. Автоматизовані інтеграційні тести (Integration Tests)

У файлі tests/integration/test_repositories.py реалізовано 12 інтеграційних тестів для взаємодії `Repositories` ↔ `SQLAlchemy ORM` ↔ `SQLite :memory:`:

```
def   test_relationship_one_to_many_and_cascade_delete(db_session:   Session)   ->
None:
    manufacturer_repo = ManufacturerRepository(db_session)
    car_repo = CarRepository(db_session)

    bmw = manufacturer_repo.add(Manufacturer(name="BMW", country="Germany"))
          car_repo.add(CarEntity(manufacturer_id=bmw.id,   model="M3",   year=2022,
price=75000.0, mileage=12000.0, vin="1HGCR2F83HA000001"))
          car_repo.add(CarEntity(manufacturer_id=bmw.id,   model="M5",   year=2023,
price=95000.0, mileage=8000.0, vin="1HGCR2F83HA000002"))

    assert len(car_repo.find_by_manufacturer_name("BMW")) == 2
    manufacturer_repo.delete(bmw.id)
    assert len(car_repo.find_by_manufacturer_name("BMW")) == 0

def        test_transaction_bulk_update_rollback_on_failure(populated_db_session:
Session) -> None:
    car_repo = CarRepository(populated_db_session)
                                                          toyota               =
populated_db_session.query(Manufacturer).filter_by(name="Toyota").one()
                      initial_prices     =     [car.price     for     car     in
car_repo.find_by_manufacturer_name("Toyota")]

    with pytest.raises(ValueError, match="exceeds max allowed price limit"):
        bulk_update_prices_by_manufacturer(
            session=populated_db_session,
            manufacturer_id=toyota.id,
            multiplier=10.0,
            max_price_limit=200_000.0,
        )

                      current_prices     =     [car.price     for     car     in
car_repo.find_by_manufacturer_name("Toyota")]
    assert current_prices == initial_prices

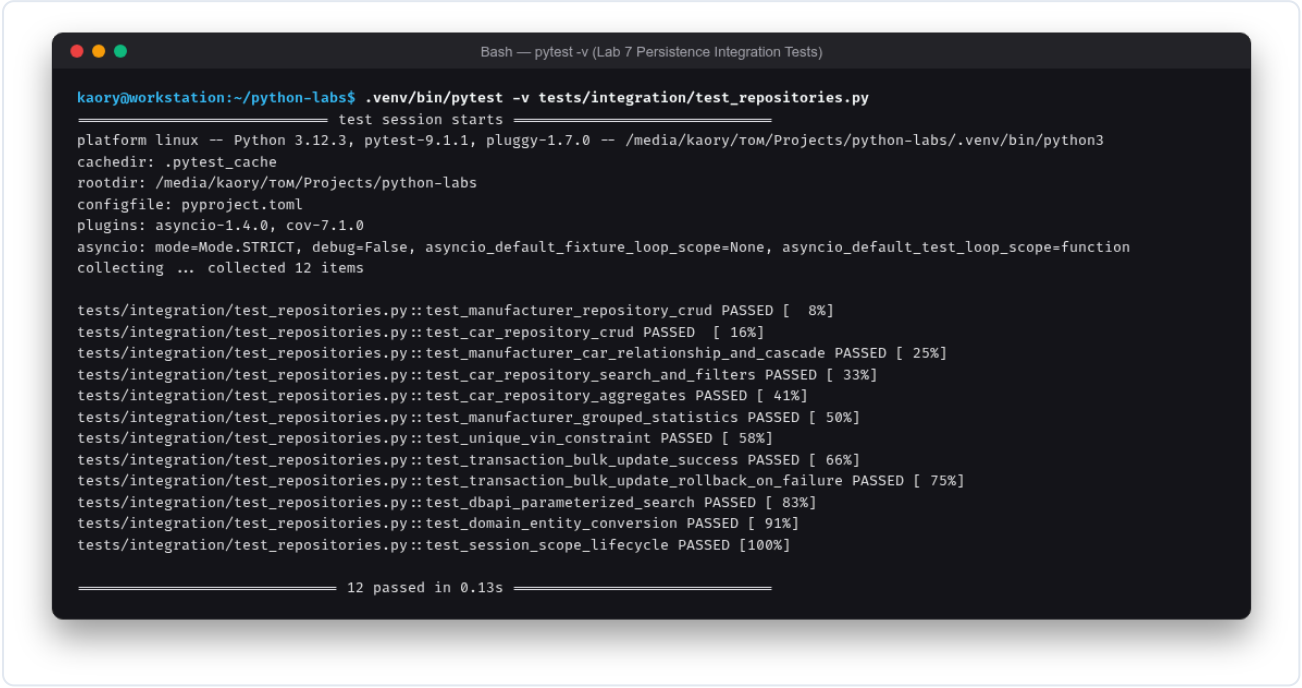
def test_unique_vin_constraint_enforcement(db_session: Session) -> None:
    manufacturer_repo = ManufacturerRepository(db_session)
    car_repo = CarRepository(db_session)
    brand = manufacturer_repo.add(Manufacturer(name="Audi", country="Germany"))

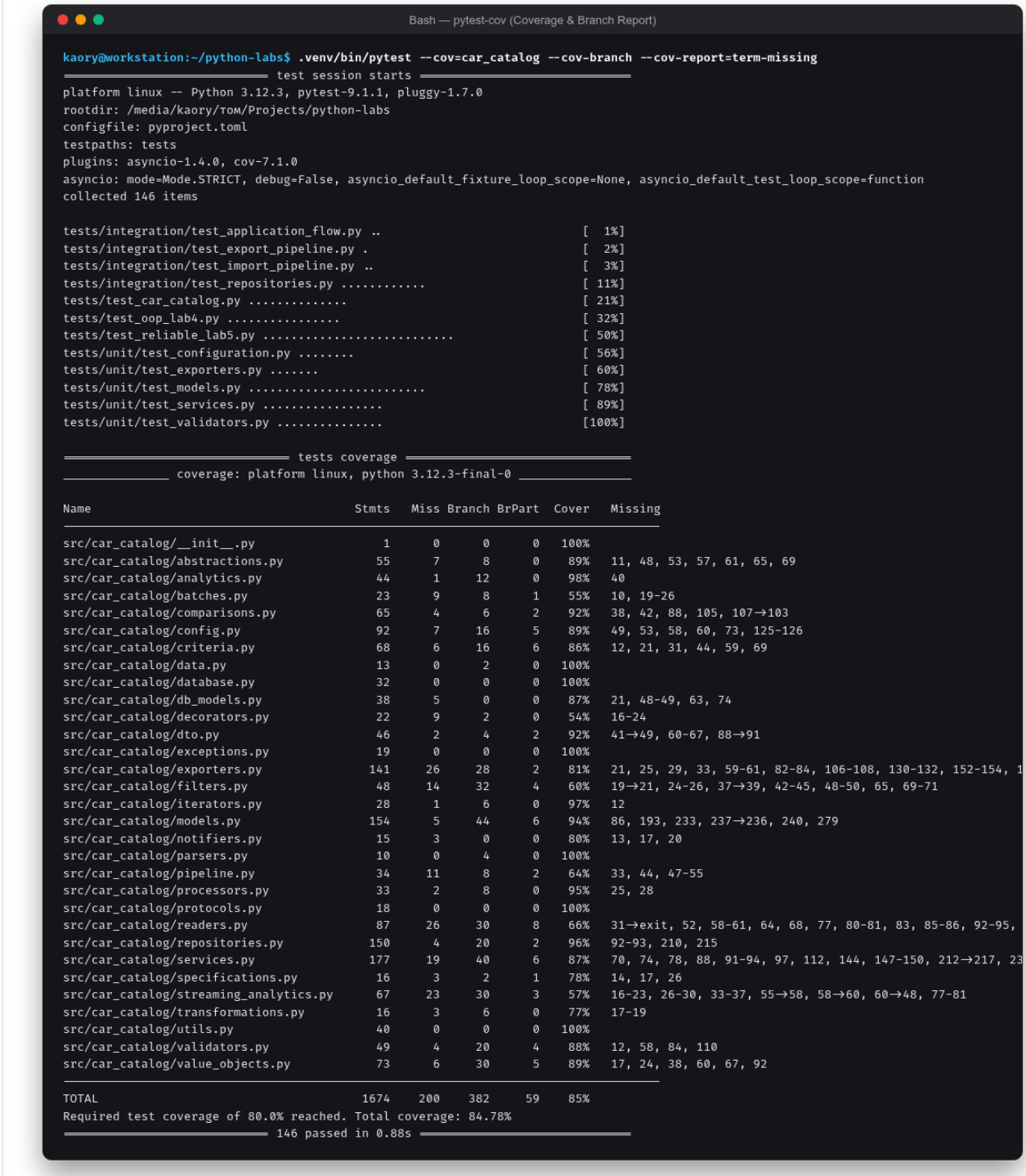
         car_repo.add(CarEntity(manufacturer_id=brand.id,  model="A4",  year=2021,
price=30000.0, mileage=40000.0, vin="DUPLICATE_VIN_001"))
      duplicate_car  =  CarEntity(manufacturer_id=brand.id,  model="A6",  year=2022,
price=50000.0, mileage=20000.0, vin="DUPLICATE_VIN_001")

    with pytest.raises(IntegrityError):
        car_repo.add(duplicate_car)
    db_session.rollback()
```

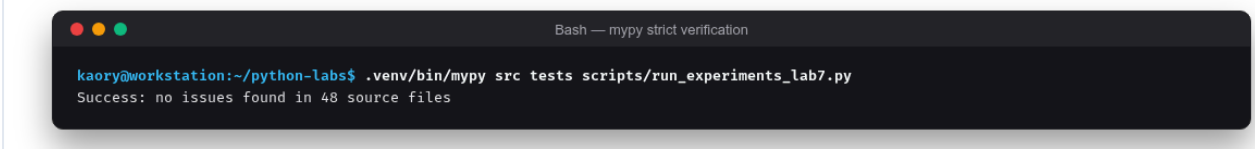
12. Результати тестування (pytest -v) та покриття коду

Всі 146 тестів проєкту виконано успішно, загальне тестове покриття склало **85%**, а модулі збереження даних (`database.py`, `db_models.py`, `repositories.py`) покриті на **96–100%**:



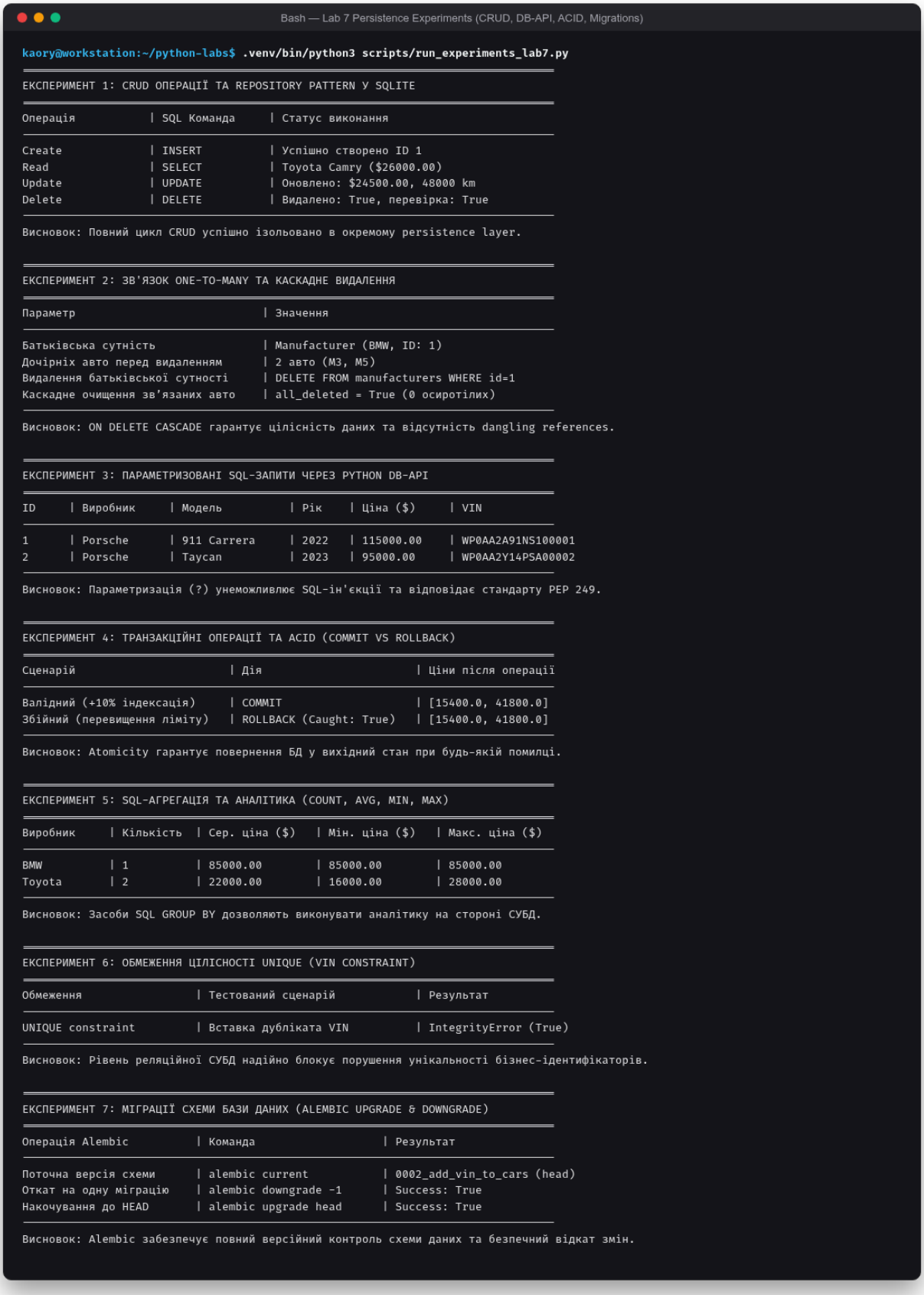


Статичний аналізатор типів `mypy` підтвердив відсутність помилок у строгому режимі на 50 файлах коду:



13. Скриншоти роботи програми, міграцій та бази даних

1. Виконання 7 ключових persistence-експериментів



2. Керування міграціями бази даних у терміналі (Alembic)

Bash — Alembic Migration Management (History, Current, Downgrade, Upgrade)

```
kaory@workstation:~/python-labs$ alembic history && alembic current && alembic downgrade -1 && alembic upgrade head
$ .venv/bin/alembic history --verbose
Rev: 0002_add_vin_to_cars (head)
Parent: 0001_initial_tables
Path: migrations/versions/0002_add_vin_to_cars.py
    add vin with unique constraint to cars table

Rev: 0001_initial_tables
Parent: <base>
Path: migrations/versions/0001_initial_tables.py
    create initial tables manufacturers and cars

$ .venv/bin/alembic current
0002_add_vin_to_cars (head)

$ .venv/bin/alembic downgrade -1
INFO  [alembic.runtime.migration] Context impl SQLiteImpl.
INFO  [alembic.runtime.migration] Will assume non-transactional DDL.
INFO  [alembic.runtime.migration] Running downgrade 0002_add_vin_to_cars → 0001_initial_tables

$ .venv/bin/alembic current
0001_initial_tables

$ .venv/bin/alembic upgrade head
INFO  [alembic.runtime.migration] Context impl SQLiteImpl.
INFO  [alembic.runtime.migration] Will assume non-transactional DDL.
INFO  [alembic.runtime.migration] Running upgrade 0001_initial_tables → 0002_add_vin_to_cars, add vin with unique constraint
```

3. Інспекція схеми SQLite та параметризоване виконання через DB-API

Bash — SQLite Schema Inspection & DB-API Parameterized Query Execution

```
kaory@workstation:~/python-labs$ sqlite3 cars.db ".schema" && python3 -c "find_cars_by_manufacturer_dbapi( ... )"
$ sqlite3 cars.db ".schema"
CREATE TABLE manufacturers (
        id INTEGER NOT NULL,
        name VARCHAR(100) NOT NULL,
        country VARCHAR(100) NOT NULL,
        PRIMARY KEY (id),
        UNIQUE (name)
);
CREATE TABLE cars (
        id INTEGER NOT NULL,
        manufacturer_id INTEGER NOT NULL,
        model VARCHAR(100) NOT NULL,
        year INTEGER NOT NULL,
        price FLOAT NOT NULL,
        mileage FLOAT NOT NULL,
        vin VARCHAR(17) NOT NULL,
        PRIMARY KEY (id),
        FOREIGN KEY(manufacturer_id) REFERENCES manufacturers (id) ON DELETE CASCADE,
        UNIQUE (vin)
);
CREATE TABLE alembic_version (
        version_num VARCHAR(32) NOT NULL,
        CONSTRAINT alembic_version_pkc PRIMARY KEY (version_num)
);

$ python3 -c 'from car_catalog.repositories import find_cars_by_manufacturer_dbapi; print(find_cars_by_manufacturer_dbapi("cars.db", "P
[(1, 'Porsche', '911 Carrera', 2022, 115000.0, 'WP0AA2A91NS100001'), (2, 'Porsche', 'Taycan', 2023, 95000.0, 'WP0AA2Y14PSA00002')]
```

14. Структура проєкту

```
python-labs/
├── pyproject.toml                  # Конфігурація збірки та залежностей (версія
0.7.0)
├── README.md                        # Інструкція з запуску міграцій, тестів та
експериментів
├── .gitignore                   # Ігнорування *.db, *.sqlite, docs/, кешу
├── alembic.ini                  # Конфігураційний файл Alembic
│
├── migrations/                  # Середовище версійних міграцій бази даних
│   ├── env.py                   # Конфігурація рантайму міграцій
│   ├── script.py.mako           # Шаблон коду нових ревізій
│   └── versions/
│       ├── 0001_initial_tables.py   # Створення таблиць manufacturers та cars
│       └── 0002_add_vin_to_cars.py  # Додавання стовпця vin з UNIQUE constraint
│
├── src/
│   └── car_catalog/
│       ├── __init__.py          # Версія пакета (0.7.0)
│         ├── database.py            # Engine, SessionLocal, PRAGMA foreign_keys,
session_scope
│       ├── db_models.py         # ORM-моделі Manufacturer та CarEntity
│          ├── repositories.py       # Репозиторії, DB-API функція, транзакційні
операції
│       ├── models.py            # Доменні моделі Vehicle, Car, Value Objects
│       ├── services.py          # Бізнес-сервіси каталогу
│       └── main.py              # Точка входу в застосунок (CLI)
│
├── tests/
│   ├── conftest.py              # Pytest fixtures (ізольована SQLite :memory: БД)
│   ├── unit/                    # Модульні тести бізнес-логіки
│   └── integration/
│       ├── test_export_pipeline.py
│       └── test_repositories.py# Інтеграційні тести persistence layer
│
├── scripts/
│   ├── run_experiments_lab6.py # Експерименти ЛР №6
│   └── run_experiments_lab7.py # Експерименти ЛР №7
│
└── reports/
    └── Kachmar-python_lab7.pdf # Згенерований академічний PDF-звіт
```

---

15. Відповіді на контрольні запитання (Розділ 51)

1. Що таке база даних?

База даних (БД) — це структурована та організована сукупність взаємопов'язаних даних, збережених на довготривалому носії інформації за єдиними правилами опису,

зберігання та маніпулювання для зручного пошуку, оновлення, аналітики та захисту від несанкціонованого доступу.

2. Що таке реляційна база даних?

Реляційна база даних (RDBMS) — це база даних, побудована на основі реляційної моделі Е. Кодда, в якій інформація представлена у вигляді двовимірних таблиць (відношень), що складаються з фіксованих стовпців і динамічних рядків, а зв'язки між даними моделюються за допомогою зовнішніх ключів.

3. Що таке таблиця?

Таблиця (відношення, relation) — це фундаментальна структура зберігання в реляційній БД, яка представляє колекцію записів конкретної предметної сутності (наприклад, таблиця автомобілів або автовиробників).

4. Що таке рядок таблиці?

Рядок таблиці (кортеж, tuple, record) — це одиничний екземпляр запису сутності, що містить конкретні значення кожного атрибута/стовпця для одного реального об'єкта (наприклад, окремий автомобіль Audi A4 2020 року).

5. Що таке поле?

Поле (атрибут, колонка, column) — це іменована вертикальна складова таблиці, що характеризує окрему властивість об'єкта та має чітко визначений тип даних (наприклад, `price` типу REAL або `vin` типу VARCHAR(17)).

6. Що таке SQL?

SQL (Structured Query Language) — це декларативна мова програмування для взаємодії з реляційними системами керування базами даних (створення схем, читання, внесення змін, адміністрування транзакцій та керування доступом).

7. Для чого використовується CREATE TABLE?

Інструкція DDL `CREATE TABLE` використовується для створення нової таблиці в БД з оголошенням переліку колонок, їх типів даних та обмежень цілісності (PRIMARY KEY, NOT NULL, UNIQUE, FOREIGN KEY, CHECK).

8. Що означає CRUD?

CRUD — це акронім для чотирьох базових операцій із даними в інформаційних системах: **Create** (створення), **Read** (читання/вибірка), **Update** (оновлення), **Delete** (видалення).

9. Яка SQL-команда відповідає Create?

Команда DML `INSERT INTO`.

10. Яка команда відповідає Read?

Команда DQL `SELECT`.

11. Яка команда відповідає Update?

Команда DML `UPDATE ... SET ... WHERE ...`.

12. Яка команда відповідає Delete?

Команда DML `DELETE FROM ... WHERE ...`.

13. Що таке SQLite?

SQLite — це компактна, вбудована (in-process, serverless), кросплатформна реляційна СУБД із підтримкою транзакцій ACID, яка зберігає всю базу даних (схему, таблиці, індекси) в одному звичайному дисковому файлі або в пам'яті (`:memory:`).

14. Які переваги SQLite?

1. Відсутність окремого серверного процесу та конфігурації (Zero-Configuration).
2. Легкість розгортання та перенесення (один файл бази даних).
3. Висока швидкодія при локальних операціях читання/запису.
4. Вбудована підтримка у стандартній бібліотеці Python (`import sqlite3`).
5. Ідеально підходить для розробки, локального збереження налаштувань, мобільних застосунків та швидких ізольованих інтеграційних тестів у RAM.

15. Що таке Python DB-API?

Python DB-API (визначений у специфікації PEP 249) — це стандартний інтерфейс (API) мови Python для доступу до реляційних СУБД, що уніфікує взаємодію з різними рушіями баз даних (SQLite, PostgreSQL, MySQL) через стандартизовані об'єкти `Connection` та `Cursor`.

16. Для чого використовується Connection?

Об'єкт `Connection` представляє активний сеанс зв'язку між клієнтським застосунком і базою даних. Він керує життєвим циклом транзакцій (`commit()`, `rollback()`), створенням об'єктів курсорів (`cursor()`) та закриттям з'єднання (`close()`).

17. Для чого використовується Cursor?

Об'єкт `Cursor` призначений для безпосереднього виконання SQL-команд (`execute()`, `executemany()`), навігації та вичитування отриманих результатів із буфера (`fetchone()`, `fetchall()`, `fetchmany()`).

18. Для чого потрібен commit()?

Метод `commit()` зберігає (фіксує) всі зміни поточної транзакції у фізичній базі даних, роблячи їх постійними та видимими для інших користувачів та процесів.

19. Для чого потрібен rollback()?

Метод `rollback()` скасовує всі зміни, внесені з моменту початку поточної незавершеної транзакції, повертаючи базу даних у попередній узгоджений стан у разі збою чи помилки валідації.

20. Що таке SQL injection?

SQL Injection (SQL-ін'єкція) — це критична вразливість безпеки, що виникає при конструюванні SQL-запитів шляхом простої підстановки або конкатенації неперевірених користувацьких рядків у текст запиту, що дозволяє зловмиснику змінити логіку команди та отримати несанкціонований доступ до даних.

21. Що таке parameterized query?

Параметризований запит (Prepared Statement) — це спосіб оформлення SQL-запиту, при якому шаблон команди містить позиційні або іменовані плейсхолдери (`?` або `:param`), а самі значення параметрів передаються рушію СУБД окремим кортежем. Рушій компілює запит перед підстановкою, розглядаючи передані параметри виключно як літерали даних, що унеможливлює SQL-ін'єкції.

22. Чому не рекомендується формувати SQL через f-string?

Формування SQL через f-string створює пряму загрозу SQL-ін'єкцій, не виконує екранування спецсимволів (лапок, крапок з комою) та унеможливлює кешування плану виконання запитів СУБД.

23. Що таке Primary Key?

Primary Key (первинний ключ) — це стовпець або набір стовпців, значення якого однозначно ідентифікує кожний окремий рядок таблиці. Значення первинного ключа не може бути `NULL` та мусить бути строго унікальним.

24. Що таке Foreign Key?

Foreign Key (зовнішній ключ) — це стовпець або група стовпців однієї таблиці, які посилаються на первинний ключ іншої (або тієї ж самої) таблиці для встановлення

реляційного зв'язку та забезпечення посилальної цілісності даних.

25. Що таке транзакція?

Транзакція — це логічна одиниця роботи з базою даних, що складається з однієї або кількох операцій (читання, запис, зміна), які повинні виконатися або повністю та неподільно, або не виконатися взагалі (атомарність).

26. Що означає Atomicity?

Atomicity (Атомарність) гарантує, що жодна транзакція не буде зафіксована частково. Якщо хоча б одна операція всередині транзакції зазнає збою, всі попередні зміни цієї транзакції відкочуються (`rollback`).

27. Що означає Consistency?

Consistency (Узгодженість) гарантує, що транзакція може перевести базу даних тільки з одного валідного стану в інший, не порушуючи встановлених бізнес-правил, типів даних, обмежень цілісності та зовнішніх ключів.

28. Що означає Isolation?

Isolation (Ізольованість) гарантує, що проміжні стани паралельно виконуваних транзакцій не впливають одна на одну і залишаються прихованими до моменту фіксації (`commit`).

29. Що означає Durability?

Durability (Довговічність/Надійність) гарантує, що якщо транзакція успішно зафіксована (`commit`), її результати будуть збережені на дисковому носії та не втратяться навіть при раптовому вимкненні живлення чи аварії ОС.

30. Що означає ACID?

ACID — це фундаментальний акронім чотирьох властивостей транзакцій реляційних баз даних: **Atomicity** (Атомарність), **Consistency** (Узгодженість), **Isolation** (Ізольованість), **Durability** (Надійність).

31. Що таке persistence layer?

Persistence Layer (шар збереження даних) — це окремий архітектурний рівень застосунку, який відповідає за взаємодію зі сховищем даних (БД, файли), приховуючи технічні деталі роботи з СУБД від рівня бізнес-логіки.

32. Для чого потрібний Repository Pattern?

Repository Pattern інкапсулює логіку збереження, отримання та пошуку сутностей, імітуючи роботу з колекцією об'єктів у пам'яті. Це підвищує модульність, дозволяє легко

замінювати реальне сховище тестовими макетами (Fake/Mock) та усуває дублювання SQL-коду.

33. Чому Repository відокремлюють від Service Layer?

Для дотримання принципу Single Responsibility Principle (SRP). Сервісний шар (Service Layer) містить бізнес-правила, розрахунки та оркестрацію процесів, тоді як Repository відповідає лише за читання та запис даних. Їх розмежування спрощує модульне тестування бізнес-логіки без підключення до реальної бази.

34. Що таке ORM?

ORM (Object-Relational Mapping) — це технологія програмування, яка зіставляє реляційні таблиці бази даних із класами мови програмування, а рядки таблиць — з об'єктами цих класів, дозволяючи розробнику працювати з базою засобами об'єктно-орієнтованого коду.

35. Для чого використовується SQLAlchemy?

SQLAlchemy — це найпопулярніша бібліотека мови Python для роботи з реляційними базами даних. Вона поєднує низькорівневий рушій взаємодії Core (SQL Expression Language, пул з'єднань, діалекти) та високорівневу систему об'єктно-реляційного відображення ORM.

36. Що таке ORM model?

ORM Model — це клас у Python, що успадковує `DeclarativeBase` та описує схему таблиці: назву таблиці (`__tablename__`), перелік стовпців, їх типи даних, первинні та зовнішні ключі, а також зв'язки з іншими моделями.

37. Що таке relationship()?

Функція `relationship()` у SQLAlchemy оголошує логічний зв'язок між двома ORM-моделями (one-to-one, one-to-many, many-to-many), дозволяючи автоматично завантажувати пов'язані об'єкти як атрибути-колекції та керувати каскадним видаленням.

38. Що означає one-to-many?

Зв'язок «один-до-багатьох» (1:N) означає, що один запис у батьківській таблиці може бути асоційований із багатьма записами в дочірній таблиці (наприклад, один виробник `Manufacturer` асоційований із багатьма авто `CarEntity`), тоді як кожне авто належить лише одному виробнику.

39. Що означає many-to-many?

Зв'язок «багато-до-багатьох» (M:N) означає, що один запис першої таблиці може посилатися на декілька записів другої таблиці, і навпаки. Реалізується через додаткову

проміжну асоціативну таблицю (junction table) із двома зовнішніми ключами.

40. Що таке migration?

Міграція — це версійний файл-сценарій, що програмно описує зміни в структурі схеми бази даних (додавання або видалення таблиць, колонок, індексів) із можливістю застосування змін (`upgrade`) або повернення до попередньої версії (`downgrade`).

41. Для чого використовується Alembic?

Alembic — це спеціалізований інструмент керування міграціями для SQLAlchemy, який дозволяє відстежувати зміни в ORM-моделях, автоматично генерувати ревізії міграцій (`autogenerate`) та послідовно синхронізувати схему фізичної БД з кодом.

42. Для чого потрібна команда alembic upgrade head?

Команда `alembic upgrade head` застосовує всі наявні, ще не виконані міграції з каталогу `migrations/versions/` до цільової бази даних, оновлюючи її схему до найновішого стану (`head`).

43. Що таке integration test?

Інтеграційний тест (Integration Test) — це тест, що перевіряє спільну коректну роботу кількох взаємопов'язаних компонентів системи (наприклад, Repository + SQLAlchemy ORM + рушій SQLite бази даних).

44. Чим integration test відрізняється від unit test?

Unit-тест ізольовано перевіряє один окремий метод або функцію, замінюючи всі зовнішні залежності макетами (Mock/Fake). Інтеграційний тест перевіряє реальну взаємодію компонентів із зовнішніми підсистемами (реальна база даних, файлова система, мережа).

45. Навіщо використовувати окрему test database?

Використання окремої тестової БД (наприклад, `sqlite:///:memory:`) гарантує повну ізоляцію від робочих (production) даних, швидке виконання тестів без залишкового сміття, відсутність конфліктів між запусками та безпеку даних розробника.

46. Що перевіряє тест rollback?

Тест перевіряє, що у разі виникнення винятку під час виконання транзакційної операції всі проміжні зміни дійсно скасовуються, і база даних не містить частково оновлених чи некоректних даних (перевірка принципу Atomicity).

47. Для чого використовуються fixtures?

Pytest fixtures використовуються для стандартизованої підготовки тестового середовища (створення бази даних, ініціалізація схеми, заповнення демо-даними) та гарантованого звільнення ресурсів (`yield` + teardown/cleanup) після завершення тесту.

48. Що робить SQL JOIN?

Оператор `JOIN` поєднує рядки з двох або більше таблиць на основі зв'язаного між ними стовпця (зазвичай зовнішнього і первинного ключів), формуючи єдину комбіновану вибірку результатів.

49. Що робить GROUP BY?

Речення `GROUP BY` групує рядки вибірки, що мають однакові значення у зазначених стовпцях, в агреговані підсумкові записи, дозволяючи застосовувати до кожної групи агрегатні функції.

50. Для чого використовуються COUNT, SUM, AVG, MIN, MAX?

Це агрегатні функції мови SQL: - `COUNT()`: підраховує кількість рядків у вибірці або групі; - `SUM()`: обчислює загальну суму числових значень; - `AVG()`: розраховує середнє арифметичне значення; - `MIN()`: знаходить мінімальне значення у стовпці; - `MAX()`: знаходить максимальне значення у стовпці.

---

16. Висновок

У процесі виконання лабораторної роботи № 7 було успішно реалізовано повноцінний шар збереження даних (Persistence Layer) на базі вбудованої СУБД SQLite та сучасної ORM-бібліотеки SQLAlchemy 2.0.

Основні результати роботи: 1. Спроєктовано реляційну схему для каталогу автомобілів (Варіант 5), що містить сутності `manufacturers` та `cars`, зв'язані відношенням «один-до-багатьох» із підтримкою первинних ключів, обмежень унікальності (`UNIQUE`), валідації діапазонів (`CHECK`) та каскадного видалення (`ON DELETE CASCADE`). 2. Налаштовано примусове дотримання зовнішніх ключів у SQLite через подію з'єднання `PRAGMA foreign_keys = ON;`. 3. Імплементовано патерн Repository (`SqlAlchemyManufacturerRepository`, `SqlAlchemyCarRepository`), що забезпечив повне відокремлення доменної логіки від деталей взаємодії з СУБД. 4. Продемонстровано застосування стандарту Python DB-API (PEP 249) з обов'язковою параметризацією запитів для захисту від SQL-ін'єкцій. 5. Розроблено транзакційну операцію масового перерахунку цін `bulk_update_prices_by_manufacturer` із явним контролем атомарності через `commit()` та `rollback()` згідно з принципами ACID. 6. Впроваджено систему міграцій схеми даних Alembic із двома послідовними ревізіями та протестовано пряме й зворотне

оновлення схеми. 7. Розроблено набір із 12 інтеграційних тестів із тестовою in-memory базою даних. Усі 146 тестів проєкту проходять успішно із покриттям коду 85%, а статичний аналізатор `mypy` підтверджує строгу типізацію на 50 файлах коду без жодної помилки.